

MailFlow

Suivi et relance du courrier

Document de passation et plan de A à Z

Version 1.0 · 8 septembre 2026

Client : opérateur minier (dossier AA MINING)

Maître d'œuvre : SAMKO Group

État du système, décisions non négociables, pièges déjà rencontrés,
plan par phases, tests en local et tests en production.

Sommaire

0. Comment se servir de ce document	3
1. L'objectif final	3
2. Le vocabulaire du projet	4
3. Les décisions non négociables	5
4. Comment le système fonctionne	7
5. L'état exact au 8 septembre 2026	9
6. Les pièges déjà rencontrés	11
7. Le plan de A à Z	12
8. Les tests en local	14
9. Les tests en production	16
10. Ce qui reste à décider	18
11. Journal des sessions	18
Annexe · Les variables de configuration	19
Annexe · Les seize codes de surveillance	20
Annexe · Les autres documents du projet	20

0. Comment se servir de ce document

Ce fichier existe pour une raison précise : **le fil se perd entre les sessions de travail**. Une personne, ou un assistant, qui reprend le projet sans contexte va refaire des choix déjà tranchés, et parfois défaire des corrections qui ont coûté une demi-journée à trouver.

Si vous êtes la personne qui pilote

Lisez la section 1 (l'objectif), puis la section 5 (où on en est), puis la section 10 (ce qu'on attend de vous). Le reste se consulte au besoin.

Si vous êtes un assistant qui reprend le travail

Lisez **la section 3 et la section 6 avant de toucher au code**. Elles contiennent les décisions déjà arbitrées et les pièges déjà payés. Le code, vous savez le lire ; ces deux sections, non, parce qu'elles ne sont écrites nulle part ailleurs.

Trois consignes qui valent pour tout le projet :

1. **Ne re-arbitrez pas une décision de la section 3** sans que la personne qui pilote vous l'ait explicitement demandé. Chacune a une raison écrite à côté.
2. **Ne dites jamais qu'une chose fonctionne sans l'avoir fait tourner**. Ce projet a une commande de preuve pour chaque brique (section 8). Utilisez-les.
3. **Complétez la section 11 en fin de session**. C'est ce qui permet à la suivante de savoir où reprendre.

1. L'objectif final

Le problème

Une entreprise reçoit du courrier électronique qui **appelle une réponse** : une administration qui pose une question, un client qui attend un devis, un transitaire qui bloque un conteneur. Une partie de ces messages reste sans réponse, non par mauvaise volonté, mais parce que personne ne tient le compte. On s'en aperçoit quand le délai est passé.

Le dispositif précédent était un tableau tenu à la main. Il a cessé d'être tenu, et personne ne s'en est aperçu avant la revue du vendredi suivant.

Ce que le système doit faire

En une phrase : **savoir en permanence quel courrier attend une réponse, qui doit la donner, depuis combien de temps, et le rappeler tout seul jusqu'à ce que la réponse parte**.

En quatre temps :

Temps	Ce qui se passe
Capter	Lire la boîte, écarter le bruit, inscrire au registre ce qui appelle une réponse
Attribuer	Ranger le dossier dans une catégorie et désigner qui doit répondre
Relancer	Rappeler la personne responsable aux échéances prévues, en jours ouvrés, aux heures ouvrables
Détecter et clore	Reconnaître la réponse quand elle part, arrêter les relances, archiver

À quoi on saura que c'est réussi

Trois mesures, à établir sur les deux premières semaines d'exploitation :

- **Aucun courrier engageant sans responsable au-delà de quatre heures ouvrées.**
- **Aucune relance envoyée sur un dossier déjà répondu.** Une seule suffit à décrédibiliser le dispositif auprès de ceux qui le subissent.
- **Le taux de bruit écarté par le filtre reste au-dessus de 60 %.** En dessous, la personne qui qualifie passe son temps sur des newsletters et abandonne.

Ce que le système ne doit surtout pas faire

- **Écrire au correspondant.** Il a écrit, il attend. La relance va à la personne de la maison qui doit répondre, jamais à l'extérieur.
- **Supprimer ou déplacer du courrier.** La lecture de la boîte est strictement en lecture seule.
- **Se taire quand il tombe en panne.** Un système de relance arrêté que tout le monde croit en marche est plus dangereux qu'un tableau papier.

2. Le vocabulaire du projet

Ces mots reviennent partout, dans le code comme dans ce document. Ils ont un sens précis ici.

Mot	Ce que c'est
Échange	Un fil de conversation, pas un message. Une ligne par affaire, quel que soit le nombre d'allers-retours. C'est l'unité que l'on suit, que l'on relance, que l'on clôt.
Correspondant	L'interlocuteur extérieur qui a écrit. Créé automatiquement à la première réception.
Responsable	La personne de la maison qui doit répondre. C'est elle que le système relance.
Captation	La lecture périodique de la boîte, et l'inscription au registre de ce qui mérite d'être suivi.
Qualification	L'acte de ranger un échange capté dans une catégorie et de lui donner un responsable. Tant qu'il n'est pas qualifié, rien ne peut être relancé .
Échéance	La date à laquelle la réponse est attendue. Calculée une seule fois et écrite en base, jamais recalculée à l'affichage. Sinon on ne peut plus expliquer pourquoi une relance est partie ce jour-là.
Jour ouvré	Lundi à vendredi, hors jours fériés saisis en base. Les délais se comptent en jours ouvrés, pas en jours calendaires.
Fenêtre d'envoi	08h00 à 18h00, heure de Bamako. Un message ne part jamais en dehors.
Relance	Un rappel adressé au responsable. Numérotée : relance 1, relance 2, relance 3. Le ton se durcit à mesure.
Escalade	Ce qui suit la dernière relance : la hiérarchie est informée qu'un dossier n'a jamais obtenu de réponse. C'est le terminus de toute chaîne de relances.
Ordonnanceur (ou moteur)	Le programme qui tourne toutes les cinq minutes, prend les travaux échus et les exécute.
Travail planifié	Une ligne en base disant « à telle heure, faire telle chose sur tel dossier ». Persisté en base et non en mémoire, pour survivre à un redémarrage.
Console	L'écran web où l'on voit les dossiers et où l'on agit dessus.
IMAP	Le protocole qui sert à lire une boîte aux lettres.

Mot	Ce que c'est
SMTP	Le protocole qui sert à envoyer un message. Ce sont deux mondes séparés : SMTP expédie, il ne range rien dans la boîte.
Message-ID	L'identifiant unique que chaque courriel porte en en-tête. C'est la clé qui permet de reconnaître un message d'un système à l'autre.
In-Reply-To / References	Les en-têtes qui disent « ce message répond à celui-là ». C'est sur eux que repose toute la détection des réponses.
UIDVALIDITY	Un numéro que le serveur IMAP attribue à un dossier. S'il change, toute la numérotation mémorisée est caduque et il faut relire depuis le début.

3. Les décisions non négociables

Chacune a été prise pour une raison, et défaire l'une d'elles casse quelque chose ailleurs. **Ne pas y revenir sans demander.**

D1. Une relance va à l'intérieur, jamais à l'extérieur

Relance.destinataire pointe sur un Utilisateur, jamais sur un Correspondant. Le correspondant attend déjà ; le relancer serait absurde et impoli.

D2. Le compteur tourne, l'envoi attend

Le délai se décompte en continu, y compris le week-end et la nuit. Mais l'envoi attend l'ouverture suivante. Le retard reste donc visible et honnête, sans que personne ne soit dérangé un dimanche.

D3. Pas d'astreinte le week-end

Un seul calendrier : lundi à vendredi, 08h-18h. Décision du client du 6 septembre 2026. Elle a fait passer le planning de 16 à 13 semaines.

D4. La seule exception au calcul en avant : la date butoir

Pour un délai imposé de l'extérieur (tutelle, douane, fiscalité), les relances se calculent **à rebours** depuis la date limite, et cette date ne se décale pas parce que personne ne travaille le week-end. Champ `Echange.dateButoir`, catégories marquées `aDateButoir`.

D5. Composer, envoyer, puis seulement enregistrer

Jamais l'inverse. Enregistrer d'abord laisserait une relance comptée qui n'est jamais partie : le dossier paraîtrait suivi alors que personne n'a rien reçu. L'ordre retenu peut produire un doublon en cas de coupure, mais un doublon est **visible** dans la boîte du destinataire. Entre une erreur visible et une erreur invisible, on choisit la visible.

Cas particulier : si l'envoi réussit et l'enregistrement échoue, le travail part en échec définitif **sans nouvelle tentative**. Réessayer enverrait un doublon, se taire laisserait la base fausse. Un humain doit regarder.

D6. Un refus n'est pas une panne

Un garde-fou qui dit non (envoi désactivé, destinataire hors liste blanche, modèle incomplet, pas de responsable) remet le travail en attente pour une heure **sans consommer de tentative**. Sinon une installation en mode observation perdrait toutes ses relances au bout de trois passages.

La nature du refus est portée par le **type de retour**, jamais devinée en comparant du texte.

D7. Refuser par défaut, trois verrous indépendants

1. MAILFLOW_ENVOI_AUTORISE doit valoir exactement oui. Ni true, ni 1, ni yes : un mot qu'on n'écrit pas par accident.
2. Chaque destinataire doit figurer dans MAILFLOW_DESTINATAIRES_AUTORISES. **Liste vide, rien ne part**. Un seul destinataire hors liste bloque l'envoi entier, sans retirer discrètement le fautif.
3. Le modèle doit être entièrement rempli. Une variable manquante annule l'envoi plutôt que de produire « Bonjour , » chez le destinataire.

D8. La marque anti-boucle

Toute relance porte l'en-tête X-MailFlow-Type, et la détection ignore tout message qui le porte. Sans cela, **chaque première relance clôturerait le dossier qu'elle relance**, parce que la relance cite le message d'origine et se dépose dans le dossier des envoyés : c'est exactement le profil d'une réponse.

D9. L'escalade ne se rabat jamais sur le responsable

Escalader vers celui qu'on relance depuis trois semaines n'est pas une escalade, c'est une quatrième relance déguisée. S'il n'y a personne de désigné, le système refuse et le signale.

D10. Lecture seule sur la messagerie

La captation ne marque rien comme lu, ne déplace rien, ne supprime rien. Aucune suppression automatique de courrier, jamais. Le système propose, un humain valide.

D11. « En retard » n'est pas un statut

C'est une dérivation : `echeance < maintenant` sur un état actif. La mettre parmi les statuts casse la répartition du tableau de bord, dont les parts ne somment plus au total. Erreur déjà commise une fois.

D12. Courriel pour ce qui est cassé, console pour ce qui demande de l'attention

Une boîte qui reçoit une alarme toutes les quatre heures pour un travail en retard devient une boîte dont on filtre les alarmes. Le tri n'est pas un raffinement, c'est ce qui garde le canal d'alerte vivant.

D13. La logique pure ne vit pas dans un module qui touche la base

Une fonction pure logée dans un fichier qui importe `lib/prisma` n'est pas testable : le chargement du module réclame `DATABASE_URL`. D'où les paires `domaine/relance.ts` (pur) et `donnees/composer-relance.ts` (lit la base), `moteur/alerte-regles.ts` (pur) et `moteur/alerte.ts` (lit et expédie). **Piège rencontré deux fois, appliquer d'emblée.**

4. Comment le système fonctionne

4.1 Les cinq couches

CONSOLE (Next.js)	Ce que les humains voient et font
ORDONNANCEUR	Cadence : que faire, et quand
EXÉCUTION (donnees/)	Écrit en base, une transaction par transition
DOMAINE (domaine/)	Décide. Aucune base, aucun réseau. Testable seul.
CONNECTEUR (connecteur/)	Parle à la messagerie : IMAP pour lire, SMTP pour envoyer

La règle qui gouverne tout : **le domaine décide, la couche d'exécution écrit, l'ordonnanceur cadence**. Aucune règle métier dans l'ordonnanceur, ce qui permettrait de le remplacer par n'importe quel autre déclencheur sans changer le comportement du système.

4.2 La vie d'un échange, pas à pas

1. Il arrive. La captation lit la boîte toutes les cinq minutes et applique le filtre d'exclusion. Un message écarté est compté, avec sa raison, pour permettre de calibrer le filtre. Un message retenu crée un *Correspondant* s'il est inconnu, un *Echange* au statut *A_QUALIFIER*, et un *Message*.

2. On le qualifie. Un humain, depuis la console, lui donne une catégorie et un responsable. Le système calcule alors l'échéance et planifie la première relance. **C'est le seul geste que le système ne sait pas faire seul aujourd'hui**, et c'est le goulot d'étranglement actuel (voir section 7, phase 6).

3. Il attend. Statut *EN_ATTENTE*. Le compte à rebours tourne.

4. On le relance. À l'échéance, si l'on est dans la fenêtre d'envoi, le moteur compose un message depuis le modèle de la catégorie, l'envoie au responsable (ou à son suppléant si une absence est déclarée), puis enregistre. Statut *RELANCE*. La relance suivante, ou l'escalade, est planifiée dans la foulée.

5. Il est escaladé. Quand les relances de la catégorie sont épuisées, une note part vers la personne désignée sur la catégorie. Statut *ESCALADE*. Tous les travaux en attente sont annulés.

6. La réponse est détectée. La captation lit aussi le dossier des envoyés. Un message sortant qui cite un message connu, et qui ne porte pas la marque *X-MailFlow-Type*, vaut réponse. Statut *REPONDU*, toutes les relances en attente sont annulées, un archivage est planifié.

7. Il est archivé. Le fil complet est déposé sous forme de fichier JSON, l'URL est enregistrée, statut *ARCHIVE*.

4.3 Les huit statuts

Statut	Sens
A_QUALIFIER	Capté, engagement non tranché
EN_ATTENTE	Responsable désigné, compte à rebours lancé
RELANCE	Au moins une relance envoyée, toujours sans réponse
ESCALADE	Relances épuisées, hiérarchie informée
REPONDU	Réponse détectée ou déclarée, relances arrêtées
SANS_SUITE	Décision explicite de ne pas répondre, motif obligatoire
HORS_PERIMETRE	Faux positif, alimente la liste d'exclusion

Statut	Sens
ARCHIVE	Fil déposé, dossier clos

Ils sont **mutuellement exclusifs**, ce qui permet à la répartition affichée de sommer au total.

4.4 Les cinq catégories et leurs règles

Chaque catégorie porte ses propres délais. Une règle dit : « à la Nième relance, attendre N jours ouvrés, et écrire à telle personne avec tel modèle ».

Catégorie	Relances	Délais (jours ouvrés)
Date butoir imposée (tutelle, douane, fiscalité)	1 puis escalade	3, puis 1 (calcul à rebours)
Approvisionnement, logistique, transit	2 puis escalade	1, 1, puis 1
Commercial, offres, contrats	2 puis escalade	2, 2, puis 3
Technique, exploitation	2 puis escalade	1, 2, puis 2
Administratif courant, personnel	1 puis escalade	3, puis 3

Trois modèles de message seulement : rappel courtois, rappel ferme, note d'escalade. Variables disponibles : {{numero}}, {{sujet}}, {{correspondant}}, {{organisation}}, {{recuLe}}, {{joursEcoules}}, {{echeance}}, {{lienMail}}, {{lienFiche}}.

4.5 Carte des fichiers

Fichier	Rôle	Lignes
src/domaine/echeance.ts	Calcul en jours ouvrés, fenêtre d'envoi, date butoir à rebours	388
src/domaine/cycle-echange.ts	Machine à états : quelles transitions sont permises, quels effets elles produisent	540
src/domaine/relance.ts	À qui la relance s'adresse, avec quelles variables	187
src/domaine/calendrier.ts	Charge le calendrier depuis la base	75
src/moteur/decision.ts	Un travail échu part-il, attend-il, ou est-il périmé	129
src/moteur/ordonnanceur.ts	Le cycle : prendre les travaux, les exécuter, rendre compte	722
src/moteur/atelier.ts	Connexions IMAP et SMTP du cycle, distinction refus / panne	153
src/moteur/sante.ts	Seize contrôles de surveillance	348
src/moteur/alerte-regles.ts	Quoi alerter, à quelle fréquence, sous quelle forme	166
src/moteur/alerte.ts	Lit les incidents, expédie le récapitulatif	166
src/connecteur/imap.ts	Lecture de la boîte, décodage des en-têtes	469
src/connecteur/envoi.ts	Envoi SMTP, marque anti-boucle, dépôt dans les envoyés	197
src/connecteur/garde-envoi.ts	Les trois verrous, remplissage des modèles	205
src/connecteur/normalisation.ts	Filtre du bruit, rattachement au bon fil	224
src/donnees/captation.ts	Inscription au registre, détection des réponses	394
src/donnees/executer.ts	Effets du domaine écrits en une transaction	322

Fichier	Rôle	Lignes
src/donnees/composer-reliance.ts	Charge un dossier et compose le message	258
src/donnees/tableau-de-bord.ts	Requêtes de la console	246
src/app/page.tsx	La console	850
src/app/actions.ts	Les huit actions de la console	209

4.6 Le modèle de données

16 tables, 14 énumérations. Les cinq qui comptent :

- `echange` : une ligne par conversation. Unicité sur `boite_id` + `conversation_id`. Porte le statut, l'échéance, le compteur de relances.
- `message` : chaque message du fil. `internet_message_id` unique : **c'est la base qui refuse le doublon**, pas une vérification dans le code.
- `relance` : une ligne par relance partie, avec le Message-ID du courriel envoyé. C'est la preuve.
- `travail_planifie` : la file d'attente. Lue avec `FOR UPDATE SKIP LOCKED`, ce qui permet plusieurs exécutants en parallèle sans double envoi.
- `evenement` : le journal. Table en écriture seule : on n'y modifie ni n'y supprime jamais une ligne.

Trois choses ne sont pas exprimables en Prisma et vivent dans `prisma/sql/` : les index partiels sur les états actifs, la colonne `tsvector` générée pour la recherche plein texte, et `FOR UPDATE SKIP LOCKED`.

5. L'état exact au 8 septembre 2026

5.1 Ce qui est construit et prouvé

Brique	État	Preuve
Modèle de données	Fait	16 tables, 3 migrations appliquées
Calcul des échéances	Fait	42 tests (jours ouverts, fuseaux, date butoir)
Machine à états	Fait	38 tests, 5 invariants
Console web	Fait	8 actions, recherche plein texte, port 3005
Lecture de la boîte (IMAP)	Fait	200 messages réels lus, chaînage vérifié
Filtre du bruit	Calibré à 65 %	31 tests
Envoi (SMTP)	Fait	<code>npm run envoi:essai</code>
Détection des réponses	Fait	Vérifiée sur des fils réels Exchange et Gmail
Ordonnanceur autonome	Fait	Capte et envoie seul, cycle de 0,9 s à vide
Canal d'alerte	Fait	17 tests, dont 48 passages simulés sur une même panne
Surveillance	Fait	16 codes, sortie 1 si critique

193 tests, 0 échec. Typecheck propre.

5.2 Ce qui n'est pas fait

Manque	Conséquence aujourd'hui
Qualification assistée	100 dossiers réels attendent, un par un, à trois décisions chacun. C'est le goulot d'étranglement du pilote.
Authentification	L'auteur des actions est codé en dur sur le premier administrateur. Le journal ne dit pas qui a fait quoi.
Destinataire d'escalade	Aucune catégorie n'en a. Toutes les escalades sont refusées, et signalées.
Stockage objet	L'archivage écrit dans <code>./archives/</code> en local.
Page par échange	Le lien d'une relance ouvre la liste filtrée sur le dossier, ce qui suffit pour agir, mais l'historique complet du fil n'a pas son écran.
Hébergement	Décision différée par le client. Butoir : semaine 11, sinon la mise en production glisse.
Git	Le projet n'est pas versionné.

5.3 L'état de la base au moment de la rédaction

```

échanges      : A_QUALIFIER=100  EN_ATTENTE=5  RELANCE=3
                ESCALADE=1  REPONDU=4  ARCHIVE=4
travaux       : SYNCHRO_BOITE en attente=1, terminés=7
                RELANCE en attente=6, terminés=2
                ESCALADE en attente=2
relances=12  messages=121  correspondants=41  utilisateurs=5
catégories=5  modèles=3  jours fériés=6  exclusions=5

```

Les 100 A_QUALIFIER viennent du **courrier réel** de `dec@samko.group`. Les dossiers en RELANCE, REPONDU et ARCHIVE, ainsi que les quatre utilisateurs en `@exemple-mining.ml`, viennent du jeu de démonstration (`npm run db:demo`).

5.4 Trois anomalies actuellement signalées, toutes légitimes

```

ENVOI_REFUSE      2 relances de démonstration hors liste blanche
ESCALADE_SANS_DESTINATAIRE 5 catégories sans personne vers qui escalader
SANS_PROPRIETAIRE 100 échanges à qualifier

```

Aucune n'est critique : le moteur sort en code 0.

5.5 Environnement technique

Next.js 16.3.4, React 19.2.8, Prisma 7.10.0 avec l'adaptateur `@prisma/adapters-pg`, PostgreSQL, Luxon pour les dates, `imapflow` pour la lecture, `nodemailer` pour l'envoi, `mailparser`.

Boîte de recette actuelle : `dec@samko.group`, chez Hostinger, en IMAP et SMTP. **Boîte visée en production** : `info@aamining.net` (l'entité), avec `mariamtraore@aamining.net` souvent en copie. Madame utilise `latyfatraos@gmail.com` et transfère à Barema.

6. Les pièges déjà rencontrés

Chacun a coûté du temps. Ils sont documentés ici pour ne pas être repayés.

6.1 Le piège de la boucle

Symptôme : chaque première relance clôture le dossier qu'elle relance. **Cause** : la relance cite le message d'origine et se dépose dans le dossier des envoyés. La détection cherche exactement cela. **Parade** : en-tête `X-MailFlow-Type` posé à l'envoi, lu à la détection. Vérifié après aller-retour serveur, parce qu'un en-tête peut être réécrit en chemin.

6.2 Un UIDVALIDITY par dossier, pas un par boîte

Symptôme : « renumérotation serveur » annoncée à chaque passage, cycle de 40 secondes. **Cause** : un seul nombre mémorisé pour deux dossiers distincts. Sur `dec@samko.group` : 1703584428 pour l'arrivée, 1703584430 pour les envoyés. **Conséquence** : la boîte était relue en entier toutes les cinq minutes, et l'alarme de renumérotation criait au loup en continu. **Parade** : `{uidValiditeEntrant, dernierUidEntrant, uidValiditeSortant, dernierUidSortant}`. **Cycle passé de 40 s à 0,9 s.**

6.3 Le piège du `N:*` en IMAP

En mode UID, une plage rend toujours au moins le dernier message du dossier, même s'il est antérieur à la borne demandée. Sans filtrage après coup, on retraits le même message à chaque passage.

6.4 `uid: true` va en troisième argument

`client.fetch(plage, requête, options)`. Mettre `uid` dans la requête au lieu des options fait lire la plage comme des numéros de séquence, et l'on récupère de tout autres messages.

6.5 Ne jamais construire une expression régulière dans un gabarit

`` ^${nom}:\s* réduit \s à s`` silencieusement. L'expression ne correspond jamais, et l'on obtient un faux négatif sans la moindre erreur. Cela a fait conclure à tort que la boîte ne chaînait pas ses réponses.

6.6 Ne pas se fier au type MIME déclaré

Le portail fiscal DGI met du HTML dans une partie annoncée `text/plain`. Détecter les balises plutôt que croire l'en-tête.

6.7 Le `no-reply` avec un jeton

Symptôme : 16 dossiers sur 100 retenus venaient d'adresses qui disent de ne pas répondre. **Cause** : la règle exigeait l'arobase collée au mot, or les envois transactionnels glissent un jeton entre les deux :

`no-reply-po9rzrtvfp90ulor5h7h2tw@mail.anthropie.com`. **Parade** :

`^(no-?reply|ne-pas-repondre|donotreply)[^@]*@`. **Ne pas élargir de même** `^(newsletter|news|info|...):info@aamining.net` est la boîte de l'entité.

6.8 Le battement de cœur avant le relevé de santé

Symptôme : `MOTEUR_SILENCIEUX` à chaque passage. Le moteur s'accuse d'être arrêté au moment précis où il tourne. **Cause** : la surveillance compare l'heure du dernier passage à la cadence attendue, et le cycle se relevait la santé avant d'avoir écrit son propre battement.

6.9 Ne pas diffuser l'arriéré au premier passage

Sans filigrane initial posé à maintenant, la première alerte contient des incidents vieux de trois semaines. Le destinataire apprend que ce canal parle du passé, et cesse de le lire.

6.10 Toute variable de modèle doit avoir un repli

`remplirModele` refuse un gabarit troué, et la captation IMAP ne renseigne ni l'organisation du correspondant ni le lien Outlook. Sans repli, chaque relance serait refusée sans que rien ne paraisse cassé. Vérifié par un test : « le gabarit complet se remplit sur un dossier nu ».

6.11 Pièges Prisma 7

- `url = env(...)` est **interdit** dans le bloc `datasource` (erreur P1012). L'URL va dans `prisma.config.ts`.
- Le client passe par un adaptateur : `new PrismaClient({ adapter: new PrismaPg({ connectionString }) })`.
- Le seed se déclare dans `prisma.config.ts` sous `migrations.seed`, plus dans `package.json`.
- Le point d'entrée du client généré est `<output>/client.ts`, pas le dossier racine.

6.12 Piège de la machine

`npx` échoue sur ce poste (`Cannot read properties of null (reading 'edgesOut')`). Contournement : installer dans un dossier, puis appeler `./node_modules/.bin/<binaire>`.

7. Le plan de A à Z

Chaque phase a un critère de sortie qui se **prouve par une commande**, pas par une opinion.

Phase 0 · Cadrage · TERMINÉE (6 septembre 2026)

Architecture, planning, 17 recommandations de performance. Décisions client : pas d'astreinte le week-end, messagerie Outlook, hébergement différé. **Sortie** : note d'architecture validée.

Phase 1 · Socle de données · TERMINÉE (6 septembre 2026)

16 tables, 14 énumérations, 3 migrations, index partiels, recherche plein texte. **Sortie** : `npm run db:check` passe.

Phase 2 · Domaine pur · TERMINÉE (7 septembre 2026)

Calcul des échéances, machine à états, décision d'exécution. **Sortie** : 101 tests au vert, sans base de données.

Phase 3 · Console · TERMINÉE (7 septembre 2026)

Écran de pilotage, 8 actions, recherche. **Sortie** : les 8 actions fonctionnent sur données réelles.

Phase 4 · Connecteur messagerie · TERMINÉE (7 et 8 septembre 2026)

Lecture IMAP, filtre, détection des réponses, envoi SMTP, garde-fous. **Sortie** : `npm run imap:test` et `npm run envoi:essai` passent.

Phase 5 · Moteur autonome · TERMINÉE (8 septembre 2026)

Captation et envoi branchés dans l'ordonnanceur, canal d'alerte, seize surveillances. **Sortie** : `npm run moteur:essai` passe de bout en bout.

VOUS ÊTES ICI

Le système tourne seul sur une boîte de recette. Il capte, il relance, il détecte, il alerte. Il ne peut rien relancer qu'un humain n'a pas qualifié, et 100 dossiers attendent.

Phase 6 · La qualification · À FAIRE, priorité 1

Pourquoi c'est la priorité. Le moteur ne peut relancer que ce qui a une catégorie et un responsable. Cent dossiers réels attendent, à trois décisions chacun : trois cents décisions, que personne ne prendra. Tant que ce goulot n'est pas levé, le moteur tourne à vide et l'alarme `SANS_PROPRIETAIRE` reste allumée en permanence, donc inaudible.

Travail :

- Pré-suggérer la catégorie et le responsable d'après l'historique du correspondant. Un correspondant déjà rencontré a presque toujours la même catégorie et le même interlocuteur.
- Qualification en lot : cocher plusieurs lignes, appliquer la même catégorie et le même responsable.
- Classement en lot vers `HORS_PERIMETRE`, qui alimente la liste d'exclusion.

Critère de sortie : moins de dix dossiers `A_QUALIFIER` de plus de quatre heures ouvrées, et l'alarme `SANS_PROPRIETAIRE` s'éteint.

Phase 7 · Configuration métier · À FAIRE, priorité 2

- Créer les utilisateurs réels : Barema, Madame, la personne d'escalade, avec suppléances.
- Renseigner `escaladeVers` sur les cinq catégories.
- Saisir les jours fériés maliens de l'année en cours.
- Relire les trois modèles de message avec le client : le ton engage l'entreprise.

Critère de sortie : `npm run moteur:sante` ne signale plus `ESCALADE_SANS_DESTINATAIRE`.

Phase 8 · Authentification · À FAIRE, priorité 3

Aujourd'hui l'auteur des actions est codé en dur. Le journal ne dit pas qui a qualifié, qui a classé sans suite. Pour un registre de correspondance, c'est une lacune de traçabilité.

Critère de sortie : chaque événement porte l'identité réelle de son auteur.

Phase 9 · Bascule sur la vraie boîte · À FAIRE

Voir la procédure détaillée en section 9.

Phase 10 · Observation et calibrage · 2 semaines, incompressible

Deux semaines de fonctionnement réel, en mode observation d'abord (envoi désactivé), puis en envoi réel vers une seule personne, puis vers tout le monde.

Critère de sortie : taux de bruit écarté stable au-dessus de 60 %, aucune relance sur un dossier déjà répondu.

Phase 11 · Hébergement et mise en production · À FAIRE

Décision d'hébergement due au plus tard semaine 11. Stockage objet à la place de `./archives/`. Sauvegardes. Supervision branchée sur le code de sortie du moteur.

Phase 12 · Reprise et transfert

Documentation d'exploitation, formation des utilisateurs, période de garantie.

8. Les tests en local

Dans l'ordre. Chaque commande prouve une chose précise ; si elle échoue, la suivante ne veut rien dire.

8.1 Avant tout

```
cd "Desktop/Claude Workspace/mailflow"
npm install
```

8.2 Le socle

Commande	Ce que ça prouve	Si ça échoue
<code>npm run typecheck</code>	Le code est cohérent	Lire l'erreur, c'est un problème de code
<code>npm test</code>	193 tests. Le calcul des dates, la machine à états, le filtre, les verrous, les alertes	Une règle métier est cassée. Ne pas continuer.
<code>npm run db:check</code>	La base répond, le schéma est en place, les index existent	Vérifier <code>DATABASE_URL</code> dans <code>.env</code>

Les tests ne touchent **ni la base ni le réseau**. Ils tournent en deux secondes et peuvent être lancés en permanence.

8.3 La messagerie

Commande	Ce que ça prouve
<code>npm run imap:test</code>	La boîte s'ouvre, le dossier des envoyés est trouvé, les en-têtes se décodent, le chaînage des réponses fonctionne sur du courrier réel
<code>npm run capter</code>	La captation lit et classe. Affiche le taux de bruit écarté
<code>npm run capter -- --limite=200</code>	Un passage plus large
<code>npm run capter -- --reinit</code>	Repart de zéro sur cette boîte

8.4 L'envoi

Commande	Ce que ça prouve
<code>npm run envoi:essai</code>	Les trois verrous, la connexion SMTP, l'envoi, le dépôt dans les envoyés, la survie de la marque anti-boucle après aller-retour serveur

Un vrai message part vers la première adresse de la liste blanche. C'est voulu.

8.5 Le moteur

Commande	Ce que ça prouve
<code>npm run moteur</code>	Un passage complet, puis le relevé de santé. Sortie 0 ou 1
<code>npm run moteur:sante</code>	La surveillance seule, sans rien exécuter ni envoyer
<code>npm run moteur:boucle</code>	Un passage toutes les 5 minutes, pour le développement
<code>npm run moteur:essai</code>	L'épreuve de bout en bout. Monte un dossier d'essai, envoie une vraie relance, relit la boîte, et vérifie que la relance ne s'est pas prise pour une réponse
<code>npm run moteur:essai -- --nettoyer</code>	Efface le dossier d'essai

`moteur:essai` est le test le plus important du projet. Il prouve en une commande que la chaîne entière tient, y compris le piège de la boucle. **Le rejouer après tout changement du moteur, du connecteur ou du domaine.**

8.6 La console

```
npm run dev -- -p 3005
```

Puis, sur `http://localhost:3005` :

1. La liste « À traiter maintenant » affiche les dossiers, du plus urgent au moins urgent.
2. Taper un **numéro seul** dans la recherche ouvre ce dossier et lui seul. C'est la cible du lien que porte chaque relance.
3. Le menu d'une ligne ne propose que les actions permises par l'état du dossier.
4. Qualifier un dossier calcule son échéance et planifie sa première relance.
5. La répartition des statuts somme au total affiché.

8.7 Les données de démonstration

```
npm run db:seed    # paramètres, catégories, modèles, exclusions
npm run db:demo    # 20 échanges fictifs, contexte minier
```

`db:demo` **efface** les boîtes, les jours fériés et les utilisateurs avant d'écrire. Il refuse de tourner si `NODE_ENV=production`.

8.8 Le scénario complet, à jouer sur une base neuve

```
npm run db:deploy  # applique les migrations
npm run db:seed    # jeu de données initial
npm run capter     # lit la vraie boîte
# qualifier un dossier depuis la console
npm run moteur     # le moteur prend le relais
npm run moteur:essai # preuve de bout en bout
npm run moteur:sante # aucun signalement critique attendu
```

9. Les tests en production

La règle qui gouverne cette section : **on n'allume pas l'envoi et la vraie boîte le même jour.**

Étape 1 · Préparer la boîte réelle, sans rien envoyer

```
MAILFLOW_BOITE="info@aamining.net"
MAILFLOW_IMAP_UTILISATEUR="info@aamining.net"
MAILFLOW_IMAP_MOT_DE_PASSE="..."
MAILFLOW_ENVOI_AUTORISE="false"          <-- fermé
MAILFLOW_DESTINATAIRES_AUTORISES=""      <-- vide
MAILFLOW_DOMAINES_INTERNES="aamining.net"
```

```
npm run imap:test
npm run capter -- --limite=200
```

Ce qu'on vérifie : la boîte s'ouvre, le dossier des envoyés est trouvé, le taux de bruit écarté est plausible, aucun message n'est marqué lu ni déplacé.

Rien ne peut partir : le premier verrou est fermé et la liste blanche est vide.

Étape 2 · Deux semaines d'observation

Le moteur tourne toutes les cinq minutes. Il capte, planifie, et refuse tous les envois. La surveillance affiche `ENVOI_DESACTIVE` : c'est normal et attendu.

Pendant ces deux semaines :

- Qualifier le courrier réel au fil de l'eau.
- Noter chaque faux positif et chaque faux négatif du filtre.
- Enrichir la liste d'exclusion depuis les dossiers classés `HORS_PERIMETRE`.

Critère de passage : le taux de bruit écarté se stabilise au-dessus de 60 %, et la personne qui qualifie ne trouve plus de surprise.

Étape 3 · Le transfert depuis la boîte de Madame

Madame reçoit sur `latyfatraos@gmail.com` et transmet à Barema. Il faut vérifier une chose précise :

Le transfert automatique conserve-t-il le Message-ID d'origine ?

Un renvoi (*redirect*) le conserve, et le chaînage tient. Un transfert (*forward*) crée un nouveau Message-ID et **casse la détection des réponses**.

Comment vérifier : faire envoyer un message d'essai à `latyfatraos@gmail.com`, le laisser se transférer, puis comparer le Message-ID reçu dans la boîte suivie avec celui d'origine.

Si le chaînage est cassé, la détection reposera sur le seul rapprochement par objet et correspondant, qui est moins sûr. À traiter avant la mise en service, pas après.

Étape 4 · Envoi réel, une seule personne

```
MAILFLOW_ENVOI_AUTORISE="oui"
MAILFLOW_DESTINATAIRES_AUTORISES="barema@aamining.net"
MAILFLOW_ALERTES_A="barema@aamining.net"
```


Une seule adresse. Toute relance destinée à quelqu'un d'autre est refusée, comptée, et signalée par `ENVOI_REFUSE`. On voit donc exactement ce qui serait parti, sans que cela parte.

Durée : **une semaine complète**, pour couvrir un cycle de relance entier, escalade comprise.

```
npm run moteur:essai # à rejouer sur la boîte réelle
```

Étape 5 · Ouverture au domaine

```
MAILFLOW_DESTINATAIRES_AUTORISES="@aamining.net"
```

Le domaine entier. À ce stade seulement, toutes les personnes concernées reçoivent leurs relances.

Ne jamais retirer complètement la liste blanche. Elle vaut désormais garde-fou permanent : une erreur de configuration ne pourra pas envoyer de relance à un client.

Étape 6 · La supervision

```
*/5 * * * * cd /opt/mailflow && npm run moteur >> /var/log/mailflow.log 2>&1
```

Sous Windows, une tâche du Planificateur toutes les cinq minutes.

Ce que la supervision doit surveiller :

Signal	Sens
Code de sortie 1	Au moins une anomalie critique
Absence de passage depuis 15 min	Le moteur est arrêté
Courriel MailFlow · sur l'adresse de supervision	Quelque chose est cassé

Étape 7 · La liste de contrôle avant de déclarer la mise en service

- ☐ `npm test` au vert
- ☐ `npm run moteur:essai` passe sur la boîte réelle
- ☐ `npm run moteur:sante` ne signale aucune anomalie critique
- ☐ Les jours fériés de l'année sont saisis
- ☐ Chaque catégorie a un destinataire d'escalade
- ☐ Chaque utilisateur actif a un suppléant
- ☐ `MAILFLOW_ALERTES_A` est renseigné et l'adresse est dans la liste blanche
- ☐ Une alerte de test a bien été reçue
- ☐ Les trois modèles de message ont été relus par le client
- ☐ La sauvegarde de la base est en place et une restauration a été testée
- ☐ La date d'expiration du secret est notée dans un agenda
- ☐ Le transfert depuis la boîte de Madame conserve le Message-ID

10. Ce qui reste à décider

Ces questions attendent une décision humaine. Elles ne sont pas des tâches de développement.

#	Question	Pourquoi ça bloque
1	La politique d'accès de l'application Entra n'est pas posée. L'application peut lire et écrire dans toutes les boîtes du locataire <code>samko-conseil.com</code> .	Exposition de sécurité. Le connecteur IMAP la rend inutile : la supprimer est sans doute plus simple que la restreindre.
2	Qui est le destinataire d'escalade de chaque catégorie ?	Sans lui, toutes les escalades sont refusées.
3	Faut-il garder ou écarter les « E-Impôt · Avis de courriel » ?	11 retenus sur 13. Décision de calibrage du filtre.
4	Faut-il exclure <code>catchall@*.odoo.com</code> ?	Même sujet.
5	<code>aam@samko-conseil.com</code> doit-elle devenir une boîte partagée ?	Libérerait une licence et donnerait nativement le montage « deux personnes, une boîte ». À ne faire que si personne ne s'y connecte comme boîte principale.
6	Où héberger ?	Butoir semaine 11, sinon la mise en production glisse.
7	Les 16 dossiers <code>no-reply</code> déjà captés : les classer hors périmètre ?	Le filtre est corrigé pour l'avenir, mais ces 16 restent dans la file de qualification.

11. Journal des sessions

À compléter à chaque fin de séance. Une entrée courte suffit, mais elle doit dire **ce qui a changé** et **ce qui bloque**.

6 septembre 2026

Cadrage. Pivot de périmètre : le système est pour un client opérateur minier, pas pour SAMKO. Démarrage du code. Modèle de données et schéma Prisma posés.

7 septembre 2026

Domaine pur (101 tests), console avec 8 actions, ordonnanceur avec battement de cœur. Correction d'une erreur d'analyse : j'avais annoncé que le risque multi-locataire était éliminé. C'était faux, `aamining.net` est chez Hostinger et non dans le locataire SAMKO.

7 et 8 septembre 2026

Connecteur IMAP et envoi SMTP. Détection des réponses vérifiée sur des fils réels. Filtre calibré de 13 % à 65 % de bruit écarté.

8 septembre 2026 (matin)

Moteur autonome. La captation et l'envoi sont branchés dans l'ordonnanceur. Découverte et correction du piège de l'UIDVALIDITY par dossier : cycle de 40 s ramené à 0,9 s. Ajout de `npm run moteur:essai`, preuve de bout en bout rejouable.

8 septembre 2026 (après-midi)

Canal d'alerte. Le moteur sait désormais dire qu'il va mal, par courriel, avec un silence de quatre heures par code d'alarme. Correction de trois défauts introduits le matin même : le lien mort dans chaque relance, le faux MOTEUR_SILENCIEUX, et la diffusion de l'arriéré au premier passage. Correction du filtre no-reply : 16 dossiers sur 100 passaient au travers.

Ce qui bloque : la qualification. 100 dossiers réels attendent, et le moteur ne peut rien relancer qu'il n'a pas qualifié.

(session suivante)

Date :
Ce qui a changé :
Ce qui est prouvé (commandes lancées) :
Ce qui bloque :

Annexe · Les variables de configuration

Fichier `.env`, jamais versionné. Modèle complet dans `.env.example`.

Variable	Rôle
DATABASE_URL	Connexion PostgreSQL
MAILFLOW_BOITE	L'adresse de la boîte suivie
MAILFLOW_IMAP_HOTE / _PORT / _UTILISATEUR / _MOT_DE_PASSE	Lecture de la boîte
MAILFLOW_SMTP_HOTE / _PORT	Envoi
MAILFLOW_DOSSIER_ENVOYES	Laisser vide : le script détecte le dossier et donne la valeur
MAILFLOW_ENVOI_AUTORISE	Verrou 1. Doit valoir exactement <code>oui</code>
MAILFLOW_DESTINATAIRES_AUTORISES	Verrou 2. Adresses ou domaines préfixés d'un <code>@</code> , séparés par des virgules
MAILFLOW_DOMAINES_INTERNES	Les domaines de la maison, pour que le courrier entre collègues n'entre pas au registre
MAILFLOW_ALERTES_A	Qui reçoit les alertes. Doit aussi figurer dans la liste blanche
MAILFLOW_URL_CONSOLE	Adresse de la console, citée dans les modèles de relance
MAILFLOW_CADENCE_CAPTATION_MINUTES	Intervalle entre deux lectures, défaut 5
MAILFLOW_NOM_EXPEDITEUR	Nom affiché comme expéditeur des relances

Ne jamais mettre un secret ailleurs que dans `.env`. Diagnostiquer un problème d'identifiant par sa longueur et sa forme, jamais en affichant sa valeur.

Annexe · Les seize codes de surveillance

Code	Gravité	Sens
MOTEUR_JAMAIS_PASSE	critique	L'ordonnanceur n'a jamais tourné
MOTEUR_SILENCIEUX	critique	Aucun passage depuis plus de deux cadences
BATTEMENT_ILLISIBLE	critique	Le dernier passage est illisible en base
TRAVAUX_EN_SOUFFRANCE	critique	Travaux échus depuis plus d'une heure
CAPTATION_EN_RETARD	critique	Une boîte active n'est plus lue. Les réponses ne sont plus détectées et les relances continuent
COMPTEUR_DIVERGENT	critique	Le compteur de relances ne correspond plus aux relances enregistrées
AUCUNE_BOITE	critique	Aucune boîte suivie active
ABONNEMENT_EXPIRE	critique ou attention	L'abonnement aux notifications expire
ENVOI_DESACTIVE	attention	Mode observation : des envois attendent et ne partiront pas
ENVOI_REFUSE	attention	Des envois sont refusés par un garde-fou
ESCALADE_SANS_DESTINATAIRE	attention	Une catégorie n'a personne vers qui escalader
CAPTATION_JAMAIS_FAITE	attention	Une boîte active n'a jamais été lue
TRAVAUX_EN_ECHEC	attention	Travaux en échec définitif
VERROUS_PERIMES	attention	Un exécutant s'est arrêté en cours de route
AUCUN_JOUR_FERIE	attention	Le calcul en jours ouvrés est faux
SANS_PROPRIETAIRE	attention	Échanges sans responsable au-delà du délai
ALERTES_SANS_DESTINATAIRE	attention	Anomalie critique en cours et aucune adresse de supervision
ANOMALIE_BOITE	attention	Incident relevé à la dernière lecture

Annexe · Les autres documents du projet

Fichier	Contenu
docs/exploitation.md	Exploitation de l'ordonnanceur, en détail
docs/modele-donnees.md	Le modèle de données, choix par choix
docs/acces-microsoft.md	Accès Microsoft 365, si l'on y revient
docs/essais-messagerie.md	Essais IMAP et SMTP
.env.example	Toutes les variables, commentées